

Programming Project: Graph Search

Instructions: Read through the project report and submit all answers to the red sections and your source files in the 'Projects' directory of your GitHub repository.

Introduction

1. Learning Objectives

Upon completing the project, you will be able to:

- Explain how to traverse a graph.
- Construct a depth-first search and breadth-first algorithm.
- Simulate a path-finder program.

2. Problem Statement

Can a program find a successful path from one point of a two-dimensional grid to another point?

3. Scope & Constraints

Constraint	Details
Implementation Language	C++
Allowed Libraries	'iostream', 'string', 'sstream', 'fstream', 'iomanip', 'cctype'
Required Header Files	'Object.h', 'Node.h', 'Deque.h', 'Point.h', 'Grid.h'
Helper & Sample Files	'sample.cpp'
Submission Deadline	Monday, May 25 - end of day

4. Submission Requirements

Item	Format
Completed lab report (this document)	.pdf
Source code files	.cpp, .h
Outputs	.txt

Report

1. Key Terms & Definitions

Define the following terms in your own words.

- Depth-First Search
- Breadth-First Search
- Graph
- Adjacent Vertices

2. Reiterate Question

Restate the problem statement in your own words.

3. Forecasting

Before writing any code, describe what you already know by completing the following work.

- a) What is the difference between a depth-first search and a breadth-first search?
- b) What are the essential data structures for each search algorithm?
- c) Write a standard depth-first algorithm as a list of English instructions.
- d) Write a standard breadth-first algorithm as a list of English instructions.
- e) Trace each search algorithm from S to E on the grid below. Boxes with an X are inaccessible.

	0	1	2	3	4
0	S			X	X
1	X		X	E	
2	X		X		
3				X	
4	X	X			

4. Hypothesis

Write your hypothesis using the format: "If [condition], then [predicted outcome], because [reasoning]."

5. Step-by-Step Procedure

a) Define the function: `void ajacents(const Point& p, const Grid& g, Deque<Point>& s)` that inserts all available adjacents of *p* from *g* into the front of *s*.

b) Define the function: `string pathfinder(Grid& g)` that returns a path as a string from the start position of *g* to the end position of *g* using the depth-first algorithm if there exists a path; otherwise, it returns an empty string.

c) Test Case 1 - Straight Path

Step 1: Create a *Grid* object with the dimensions 2 by 5.

Step 2: Load the grid "0000011110" into the *Grid* object.

Step 3: Make the start point and end point of the *Grid* object (0,0) and (1,4), respectively.

Step 4: Display the path generated by `pathfinder()` and the output of the `path()` method of the *Grid* object with the generated path as its argument.

d) Test Case 2 - Verification

Step 1: Create a *Grid* object with the dimensions 5 by 5.

Step 2: Load the grid "0001110100101000001011000" into the *Grid* object.

Step 3: Make the start point and end point of the *Grid* object (0,0) and (1,3), respectively.

Step 4: Display the path generated by `pathfinder()` and the output of the `path()` method of the *Grid* object with the generated path as its argument.

e) Test Case 3 - No Path

Step 1: Create a *Grid* object with the dimensions 3 by 3.

Step 2: Load the grid "011010111" into the *Grid* object.

Step 3: Make the start point and end point of the *Grid* object (0,0) and (1,2), respectively.

Step 4: Display the path generated by `pathfinder()` and the output of the `path()` method of the *Grid* object with the generated path as its argument.

6. Implementation Notes

Document notable design decisions, bugs encountered, and any deviations from the planned procedure here. Include the reason for each deviation and how it affected your implementation.

7. Code Repository

List all submitted source files with a brief description of each.

8. Output

Describe and store program output for each test case in separate .txt files.

9. Data Analysis

Complete after running your program. Analyze trends, patterns, and anomalies with specific values.

10. Conclusion

Write a concise conclusion addressing the original problem statement.